Compression d'un message : les codes de Huffman

La calculatrice et les documents sont interdits. Une fonction demandée dans une question peut être utilisée dans les questions suivantes, même si elle n'a pas été écrite. Les questions peuvent être traitées dans l'ordre souhaité, à condition d'indiquer clairement leur numéro.

Rappels concernant le langage Python

- Les fonctions `min`, `max`, `sum`, `sort`, `sorted`, `index` et `count` sont **interdites**. Sont autorisés : `len(L)`, `L.append(e)`, `L.pop()` (qui retire et renvoie le dernier élément), le tranchage et la concaténation.
- **Dictionnaires.** On admet que l'accès, l'ajout et la modification d'une clé coûtent $O(1)$. Exemple d'utilisation :

```
d = {}                  # dictionnaire vide
d['c'] = 1              # ajout de la clé 'c', associée à 1
d['c'] = d['c'] + 1     # modification : d vaut maintenant {'c': 2}
'c' in d                # True : la clé 'c' existe
for x in d:             # parcours des clés
    print(x, d[x])
```

Toute complexité demandée est une complexité temporelle dans le pire des cas, exprimée avec la notation O.

Barème indicatif : partie I sur 4 points, partie II sur 8 points, partie III sur 4 points, partie IV sur 4 points.

Introduction

Le problème. Un ordinateur stocke un texte sous forme de **bits** (des `0` et des `1`). Pour cela, on choisit un **code** : à chaque lettre, on associe un mot binaire. Le message codé est la **concaténation** des codes de ses lettres, écrits les uns après les autres, *sans séparateur*.

La façon la plus simple de faire est un **code de longueur fixe** : toutes les lettres reçoivent un code de même longueur. Mais dans un texte, certaines lettres sont beaucoup plus fréquentes que d'autres. L'idée est alors la même que celle du code Morse, où la lettre E, très fréquente en anglais, est codée par un seul signal : **plus une lettre est fréquente, plus son code est court**.

Exemple : le message `abacaba`

Ce message de 7 lettres contient `a` 4 fois, `b` 2 fois et `c` 1 fois.

	`a`	`b`	`c`	message codé	longueur
code de longueur fixe	`00`	`01`	`10`	`00 01 00 10 00 01 00`	14 bits
code de longueur variable	`1`	`01`	`00`	`1 01 1 00 1 01 1`	10 bits

Les espaces ne sont là que pour faciliter la lecture : le message réellement transmis est `1011001011`. Le second code fait gagner 4 bits sur 14.

Un bon code doit vérifier deux exigences :

- **(E1)** on doit pouvoir **décoder** le message sans ambiguïté, alors qu'il n'y a pas de séparateur entre les lettres ;

- **(E2)** le nombre total de bits du message codé doit être **le plus petit possible**.

En 1952, l'étudiant américain David Huffman a trouvé un algorithme simple qui construit un tel code. Il est encore utilisé aujourd'hui dans les formats ZIP, JPEG et MP3.

Codes et arbres. On représente un code par un **arbre de codage** : un arbre binaire dont chaque nœud a zéro ou deux fils, dont chaque feuille porte une lettre et dont chaque nœud interne porte la valeur `None` (en Python, `None` est une valeur spéciale qui signifie « aucune valeur » : un nœud interne ne porte pas de lettre, on lui donne donc l'étiquette `None`). Le **code** d'une lettre est le chemin de la racine à sa feuille : on écrit `'0'` quand on descend à gauche et `'1'` quand on descend à droite.

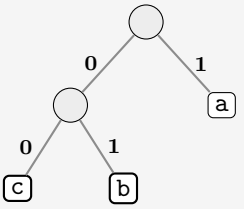
En Python, un arbre non vide est la liste `[r, g, d]` où `r` est l'étiquette de la racine et `g`, `d` les sous-arbres gauche et droit ; l'arbre vide est `[]`. Ainsi :

- une feuille portant la lettre `x` s'écrit `[x, [], []]` ;
- un nœud interne de fils `g` et `d` s'écrit `[None, g, d]` ;
- un arbre non vide `a` est une feuille si et seulement si `a[1] == []` et `a[2] == []`.

Exemple : l'arbre A du code de longueur variable

```
A = [None, [None, ['c', [], []],
                  ['b', [], []]],
           ['a', [], []]]
```

Pour trouver le code de `b`, on part de la racine, on descend à gauche (`'0'`) puis à droite (`'1'`) : le code de `b` est `'01'`.



codes : `c` → `00`, `b` → `01`, `a` → `1`

Fil rouge. Dans tout le sujet, on étudie un message de **36 lettres** comportant **5 lettres différentes**, avec les fréquences (nombres d'apparitions) suivantes :

```
F = [('a', 14), ('b', 9), ('c', 7), ('d', 4), ('e', 2)]
```

Ces fréquences peuvent aussi être rangées dans un dictionnaire, par exemple `{'a': 14, 'b': 9, ...}`.

Plan du sujet.

- **Partie I :** compter les lettres et calculer la taille du message avec un code de longueur fixe (la référence à battre) ;
- **Partie II :** utiliser un arbre de codage donné pour coder et décoder ;
- **Partie III :** construire l'arbre de Huffman à l'aide d'un tas ;
- **Partie IV :** mesurer le gain obtenu.

Partie I — Compter les lettres (4 points)

► **Q1 — Comptage des lettres.**

► **1.1** Écrire une fonction `compter_lettres(m)` qui reçoit un message `m` (une chaîne de caractères) et renvoie un dictionnaire associant à chaque lettre son nombre d'apparitions. On parcourra le message une seule fois.

Exemple

```
>>> compter_lettres("abacaba")
{'a': 4, 'b': 2, 'c': 1}
```

▶ **1.2** Donner la complexité en fonction de la longueur n du message, en la justifiant.

▶ **Q2 — Code de longueur fixe.** Pour coder $l \geqslant 2$ lettres avec des codes de même longueur, il faut b bits, où b est le plus petit entier tel que $2^b \geqslant l$.

▶ **2.1** Écrire une fonction `bits_fixe(l)` qui calcule b **sans utiliser de logarithme**.

Exemple

```
>>> bits_fixe(5)
3
>>> bits_fixe(100)
7
```

Indication : on pourra calculer les puissances successives $1, 2, 4, 8, \ldots$ en comptant les multiplications par 2, jusqu'à atteindre ou dépasser l.

▶ **2.2** Combien de bits faut-il pour coder le message du fil rouge (36 lettres, 5 lettres différentes) avec un code de longueur fixe ?

Partie II — Coder et décoder avec un arbre (8 points)

Dans toute la suite du sujet, on utilise l'arbre de codage `H` suivant (on admet que c'est l'arbre de Huffman obtenu à partir de `F`) :

```
H = [None, ['a', [], []],
           [None, ['b', [], []],
                  [None, [None, ['e', [], []], ['d', [], []]],
                         ['c', [], []]]]]
```

▶ **Q3 — Lecture de l'arbre.**

▶ **3.1** Dessiner l'arbre `H`, en indiquant `0` ou `1` sur chaque arête, comme dans l'exemple de l'arbre `A`.

▶ **3.2** Donner le code de chacune des cinq lettres.

▶ **3.3** Coder le mot `"bac"`, puis décoder, à la main, la suite de bits `"11011100111011011100"`.

▶ **Q4 — Table des codes.** On veut construire le dictionnaire des codes d'un arbre. On utilise une fonction récursive `remplir(a, chemin, d)` qui ajoute au dictionnaire `d` le code de chaque lettre de l'arbre `a`, sachant que `chemin` est le code de la racine de `a` (le chemin déjà parcouru depuis la racine de l'arbre complet) :

- si `a` est une feuille, on associe `chemin` à sa lettre ;
- sinon, on appelle `remplir` sur le fils gauche avec `chemin + '0'`, et sur le fils droit avec `chemin + '1'`.

▶ **4.1** Recopier et compléter les deux fonctions suivantes.

```
def remplir(a, chemin, d):
    if a[1] == [] and a[2] == []:
        ..........
    else:
        remplir(a[1], ..........., d)
        remplir(..........., ..........., d)

def codes(a):
    d = {}
    ..........
    return d
```

Exemple

```
>>> codes(A)
{'c': '00', 'b': '01', 'a': '1'}
```

▶ **4.2** Combien d'appels à `remplir` sont effectués pour un arbre de codage à N nœuds ?

▶ **Q5 — Coder un message.** Écrire une fonction `coder(m, d)` qui renvoie la chaîne de bits codant le message `m`, où `d` est un dictionnaire de codes : on parcourt le message et on concatène les codes de ses lettres.

Exemple

```
>>> coder("abacaba", codes(A))
'1011001011'
```

▶ **Q6 — Décoder un message.** Pour décoder, on part de la racine et on lit les bits un par un : on descend à gauche pour un `'0'`, à droite pour un `'1'`. Dès qu'on atteint une feuille, on ajoute sa lettre au résultat et on **repart de la racine**.

▶ **6.1** Écrire une fonction `décoder(a, bits)` qui renvoie le message codé par la chaîne `bits` avec l'arbre `a`.

Indication : on utilisera une variable `courant`, initialisée à `a`, qui désigne le nœud où l'on se trouve.

▶ **6.2** Donner sa complexité en fonction de la longueur de `bits`.

▶ **Q7 — Pourquoi le décodage fonctionne.** On dit qu'un code est **préfixe** si aucun code n'est le début d'un autre code. Expliquer pourquoi le code obtenu à partir d'un arbre de codage est toujours préfixe.

Partie III — Construction de l'arbre de Huffman (4 points)

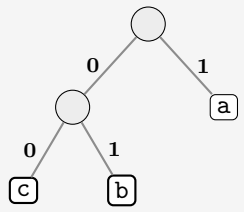
L'algorithme de Huffman. On manipule une collection d'arbres, chacun accompagné d'un **poids** : la somme des fréquences de ses feuilles.

- Au départ, chaque lettre forme un arbre réduit à une feuille, de poids sa fréquence.
- Tant qu'il reste au moins deux arbres, on retire les deux arbres de plus petits poids $p_1 \leqslant p_2$, et on les remplace par l'arbre `[None, a1, a2]`, de poids $p_1 + p_2$ (le premier retiré devient le fils gauche).
- L'arbre restant est l'arbre de Huffman.

Exemple : le message `abacaba`

	arbres retirés	arbres restants (poids)
départ	—	`c` (1), `b` (2), `a` (4)
étape 1	`c` puis `b`	`[c,b]` (3), `a` (4)
étape 2	`[c,b]` puis `a`	`[[c,b],a]` (7)

On obtient les codes `c` → `00`, `b` → `01`, `a` → `1` : on retrouve l'arbre `A` de l'introduction, et le message codé fait bien 10 bits.



Le tas-min. Pour retirer rapidement les plus petits poids, on range les couples `(poids, arbre)` dans un **tas-min** : une liste `t`, vue comme un arbre binaire presque complet, où **chaque élément a un poids inférieur ou égal à celui de ses fils**. Pour l'élément d'indice i :

- ses fils sont aux indices $2i+1$ (gauche) et $2i+2$ (droit), s'ils existent ;
- son père (pour $i > 0$) est à l'indice `(i - 1) // 2`.

Le plus petit poids est donc toujours à l'indice 0. On compare deux éléments **par leur seul poids** `t[i][0]`.

On **admet** l'existence des deux fonctions suivantes :

- `monter(t, i)`, qui fait remonter l'élément d'indice `i`, en l'échangeant avec son père, tant que son poids est strictement inférieur à celui de son père ;
- `descendre(t, i)`, qui fait descendre l'élément d'indice `i` tant qu'il a un fils de poids strictement plus petit, en l'échangeant à chaque étape avec **le plus petit** de ses fils.

▶ **Q8** En utilisant les fonctions `monter` et `descendre`, écrire les deux fonctions suivantes.

- `ajouter(t, x)` : on place `x` à la fin de `t`, puis on le fait monter.
- `retirer_min(t)` : on échange la racine avec le dernier élément, on retire ce dernier élément (c'est le minimum) avec `t.pop()`, puis on fait descendre la nouvelle racine. La fonction renvoie l'élément retiré.

▶ **Q9 — L'algorithme de Huffman.** Recopier et compléter la fonction suivante, qui reçoit une liste `L` de couples (lettre, fréquence) (avec au moins deux lettres) et renvoie l'arbre de Huffman.

```
def huffman(L):
    t = []
    for (x, f) in L:
        ajouter(t, (f, ..........))
    while ..........:
        p1, a1 = retirer_min(t)
        p2, a2 = ..........
        ajouter(t, (..........., ..........))
    return ..........
```

▶ **Q10 — Complexité.** On note l le nombre de lettres. On admet que, lors de l'exécution de `huffman`, chaque appel à `monter`, `descendre`, `ajouter` ou `retirer_min` a une complexité en $O(\log l)$. Déterminer la complexité de `huffman` en fonction de l.

Partie IV — Mesurer le gain (4 points)

Le **coût** d'un arbre de codage est le nombre total de bits du message codé :

$$\text{coût}(a) = \sum_{\text{lettres } x} f(x) \times p(x),$$

où $f(x)$ est la fréquence de x et $p(x)$ la profondeur de sa feuille, c'est-à-dire la longueur de son code.

▶ **Q11 — Coût de l'arbre H.** En utilisant les codes trouvés à la question 3.2, calculer $\text{coût}(H)$ pour les fréquences `F` du fil rouge.

▶ **Q12 — Calcul du coût.** Écrire une fonction récursive `bits_huffman(a, f, p)` qui renvoie le coût de l'arbre `a` dont la racine est à la profondeur `p`, où `f` est un dictionnaire des fréquences. L'appel initial se fait avec `p = 0`.

Indication : si `a` est la feuille d'une lettre x, son coût vaut $f(x) \times p$; sinon, c'est la somme des coûts de ses deux fils, qui sont à la profondeur $p+1$.

Exemple

```
>>> bits_huffman(H, {'a': 14, 'b': 9, 'c': 7, 'd': 4, 'e': 2}, 0)
```

doit renvoyer la valeur trouvée à la question 11.

▶ **Q13 — Gain de compression.** On appelle **gain** du code de Huffman la proportion de bits économisés par rapport au code de longueur fixe :

$$G = \frac{B_{\text{fixe}} - B_{\text{Huffman}}}{B_{\text{fixe}}},$$

où B_{fixe} et B_{Huffman} sont les nombres de bits du message codé avec le code de longueur fixe et avec le code de Huffman.

▶ **13.1** En utilisant les résultats des questions 2.2 et 11, calculer le gain G pour le fil rouge, sous forme de fraction, puis montrer qu'il est compris entre 25 % et 33 %.

▶ **13.2** En pratique, un texte est souvent stocké avec **8 bits par caractère** (code ASCII). Recalculer le gain G du code de Huffman en prenant ce codage à la place du code de longueur fixe (on doit trouver un gain supérieur à 70 %).

▶ **13.3** Écrire une fonction `gain(a, f)` qui renvoie le gain G de l'arbre de codage `a` par rapport au code de longueur fixe, où `f` est le dictionnaire des fréquences. On pourra utiliser les fonctions `bits_fixe` et `bits_huffman`.

Fin du devoir.